Experiment No.: 10

1. Aim: - Write a program to develop a GAN for Generating MNIST Handwritten Digits.

2. Objective: -

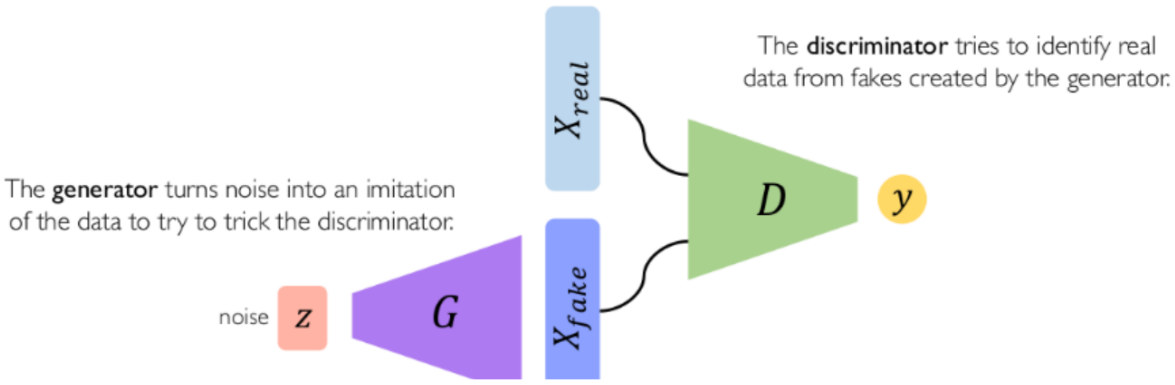
- To implement a Generative Adversarial Network (GAN) for generating handwritten digit images using the MNIST dataset.
- To understand how deep learning models can generate new data samples similar to the training dataset.

3. Outcome: -

- Students will understand the working principle of Generative Adversarial Networks.
- Students will be able to generate synthetic handwritten digit images using deep learning techniques.
- Students will learn how generator and discriminator networks interact during training.

Software Required: Python

4. Theory:



Generative Adversarial Networks (GANs) are a class of deep learning models used for generating new data samples that resemble a given dataset. GANs consist of two neural networks: the Generator and the Discriminator, which compete during training.

- **Generator:** Generates fake images from random noise.
- **Discriminator:** Evaluates whether the image is real (from dataset) or fake (generated by the generator).

During training, the generator improves his ability to produce realistic images, while the discriminator improves his ability to distinguish real images from fake ones.

The **MNIST dataset** contains grayscale images of handwritten digits from 0 to 9. It is commonly used for training and testing image processing and deep learning models.

GANs are widely used in applications such as image synthesis, data augmentation, image super-resolution, and creative AI systems.

5. Design/ Algorithm

1. Import necessary libraries such as TensorFlow, Keras, NumPy, and Matplotlib.
2. Load the MNIST dataset.
3. Normalize the image data.
4. Build the Generator network to create fake images from random noise.
5. Build the Discriminator network to classify real and fake images.
6. Combine both models to create the GAN architecture.
7. Train the discriminator with real and generated images.
8. Train the generator to fool the discriminator.
9. Repeat the training process for multiple epochs.
10. Generate and visualize synthetic handwritten digit images.

6. Step-by-Step Procedure

1. Install required Python libraries such as TensorFlow, Keras, NumPy, and Matplotlib.
2. Import necessary libraries in Python.
3. Load and preprocess the MNIST dataset.
4. Design the generator model using neural network layers.
5. Design the discriminator model for binary classification.
6. Combine both models to form a GAN architecture.
7. Train the GAN model for several epochs.
8. Generate new handwritten digit images using the trained generator.
9. Display and visualize the generated images.
10. Evaluate the quality of generated images.

7. Conclusion:

In this experiment, a Generative Adversarial Network (GAN) was developed to generate handwritten digit images using the MNIST dataset. The generator network produced synthetic images while the discriminator distinguished between real and generated images. Through adversarial training, the generator gradually improved its ability to create realistic digit images. This experiment demonstrates the power of GANs in generating new data samples and highlights their applications in image generation and deep learning.

8. Viva Question

1. What is a Generative Adversarial Network (GAN)?
2. What are the two main components of GAN?
3. What is the role of the generator in GAN?
4. What is the function of the discriminator?
5. What is the MNIST dataset?

6. What is adversarial training?
7. Why are GANs used for image generation?
8. What are some applications of GANs?

8. References

1. I. Goodfellow et al., Generative Adversarial Networks, 2014. https://arxiv.org/abs/1406.2661
2. TensorFlow Deep Learning Tutorials https://www.tensorflow.org/tutorials
3. Keras Documentation https://keras.io
4. K. P. Murphy, Machine Learning: A Probabilistic Perspective, Cambridge, MA, USA: MIT Press, 2012.